

Project Plan: JAX-Accelerated Bidirectional Parsing

This document outlines the plan for implementing a JAX-accelerated parser based on bidirectional context expansion.

Phase 1: Data Representation & 'The Catalog'

To make the 'Catalog' (currently 32k n-grams) GPU-compatible, we must vectorize it.

1.1 Context Vectorization

- * Instead of storing tuple strings (left, right), we encode them into vectors.
- * Approach: Use a lightweight embedding (e.g., small BERT/GloVe) for context windows.
- * Representation: A context c becomes a vector v_{ctx} .
- * Benefit: Solves the representation challenge regarding pattern generalization.

1.2 The Interaction Matrix (Sparse to Dense)

We construct a global map of Units fitting into Contexts.

- * Construct a frequency-weighted Adjacency Matrix A .
- * Rows = Unique Units (e.g., 'go out', 'my money').
- * Cols = Context Cluster Centroids (e.g., $k=4096$).
- * $A_{ij} = 1$ (or freq) if Unit i appears in Context Cluster j .
- * JAX Optimization: Store A as a sparse matrix (BCOO) on GPU.

Phase 2: The Expansion Kernel (JAX Core)

(Cont...)

JAX Expansion & Parsing Logic

2.2 Unit Expansion (The 'Who else?')

* Operation: Multiply the context mask m_{ctx} by the Adjacency Matrix A .

* Math:

$$V_{units} = A \cdot m_{ctx}$$

* Result: A vector where high values indicate units fitting the expanded contexts.

2.3 Energy Calculation

* We implement the energy score using differentiable ops:

$$E = -\log(|units| \times \log(|contexts| + 1))$$

* Note: We use Softmax/LogSumExp to approximate counts for differentiability.

Phase 3: The Parsing Shell (Hybrid)

3.1 Top-Down Controller (Python)

- * Manages recursive splitting of the sentence.
- * Identifies candidate spans (e.g., 'my money').

3.2 Batch Scoring (JAX)

- * Collect all candidate spans for a sentence.

(Cont...)

Architecture & Modules

Suggested Module Structure

1. data_loader.py
 - * Ingests text and builds Unit-Context counts.
 - * Runs K-Means to compress raw contexts into centroids.
2. expansion_engine.py (JAX)
 - * Contains @jax.jit functions for similarity.
 - * Holds static matrices (Adjacency A, Centroids C) in GPU memory.
3. parser.py
 - * Implements recursive logic and batches spans for the engine.

Next Steps

We will begin by mocking up the Expansion Engine to demonstrate the vector math before building the data loader.